

Assignment 2: Implement Parsers via ANTLR and JavaCC

-

CPSC 499.02: Software Analysis Winter 2026

Group 4 Members:

Elda Britu - 30158734
Collin Mtendamema - 30139450
MD Saif Al-Deen - 30197566
Rylan Laplante - 30070936

Professor Robert Walker

February 13, 2026

The ANTLR Parser

Going into creating the ANTLR parser felt relatively easy. Since we started from the lexer provided, the primary struggle came from translating the syntax from the provided textbook into proper ANTLR notation. However, that in itself was relatively simple. Rules like *classDeclaration: CLASS Identifier (EXTENDS type)? (IMPLEMENTS typeList)? classBody;* for example maps almost 1:1 to the Java textbook's BNF notation, which made translation straightforward. ANTLR's support for optional elements (?), repetition (*, +), and alternatives (|) covered every pattern needed for Java 1.2 without workarounds.

However, there were some issues when it came to correcting the grammar. For example, the original *findNewKeyword* recursed unboundedly up the tree, risking false matches on unrelated new expressions. We replaced it with a one-level parent check, since *new* is always a direct sibling of *innerCreator*. Adding onto this, it was when we began using the test cases we created that encountering some logical issues. When ran with the test cases we created, it often ended up raising an exception and closing the invocation program rather than return the "invalid" decider. It took adjusting the error catching to ensure that it would still run.

The JavaCC Parser

Observed Differences Between ANTLR and JavaCC Parsers

- ANTLR uses extended BNF with ?, *, + for optionals and repetition. It separates lexer and parser grammars cleanly. On the other hand, Java uses EBNF-style syntax with embedded Java actions, and both the lexer and parser rules live in the same .jj file.
- ANTLR's error handling involved automatic error recovery with detailed error messages. *getNumberOfSyntaxErrors()* comes in handy for programmatic checks. However, JavaCC grants more control over its error handling through try-catch blocks in the grammar, at the cost of more boilerplate code.
- Once generated, ANTLR produces clean, modular Java classes with the listener and visitor patterns built-in. The resulting parse tree is explicit and accessible via *ParseTreeWalker*. JavaCC generates a single monolithic parser class, with the semantic actions embedded directly in grammar rules. Tree construction is manual unless using an add-on.

- Debugging ANTLR code was more efficient, since it provides visual grammar debugging and parse tree inspection. The error messages are straightforward. On the other hand, the errors in JavaCC are often cryptic and point to generated code, not grammar rules.
- Overall, ANTLR had a steeper initial learning curve due to listener/visitor patterns. However, it made up for it with excellent documentation and online sources; a larger community meant more examples were available.
- JavaCC has a simpler conceptual model (grammar + actions = parser), but its documentation is sparse and outdated, and the smaller community meant fewer resources. Merely Googling 'JavaCC' resulted in it auto-correcting to 'javac'

References

Chapter 1 introduction to javacc 1.1 javacc and parser generation. (n.d.).
<https://www.engr.mun.ca/~theo/JavaCC-Tutorial/javacc-tutorial.pdf>

File: Unicode.md: Debian sources. File: unicode.md | Debian Sources. (n.d.).
<https://sources.debian.org/src/antlr4/4.9.2-3/doc/unicode.md>

Gosling, J. (2000). *The java language specification*. Addison-Wesley. pp 449-456

JavaCC. (n.d.). <https://javacc.github.io/javacc/>

Lievaart, E. (n.d.). *Erik's Java rants*. Lexical Analysis in JavaCC.
<http://eriklievaart.com/web/javacc2.html>

Ostering, B. (2015). Parser generators: ANTLR vs JavaCC. DZone.
<https://dzone.com/articles/antlr-and-javacc-parser-generators>

Parser generators: Antlr vs javacc. (n.d.-b).
<https://dzone.com/articles/antlr-and-javacc-parser-generators>

Stalla, A. (2021, November 24). *Converting from javacc to antlr*. Strumenta.
<https://tomassetti.me/converting-from-javacc-to-antlr/>

UNICAM. (n.d.-b).
<https://www.cs.unicam.it/diberardini/didattica/lpc/2009-10/slides/javacc/intro.pdf>